

TECH SPEC V2 — Catalyst Feature Flag Platform

Version: 2.0 | Status: Ready for Build | Source: docs/v2/PRD.md V1.1

Stack locked: Fastify + Prisma (Node TS) | React Vite SPA | Postgres 16 | Docker Compose | JS + Python SDKs

Mode: Fetch-on-load SDK evaluation + server fallback | Simple predictable rules | Auto-provision onboarding
> What changed from V1 (TECH_SPEC v1.0): removed push propagation. No SSE, no polling, no Redis pub/sub, no <5s gate. The SDK fetches the flag snapshot when the app loads (init), evaluates locally, and picks up any toggle on the next load/refresh.

Stack	Fastify + Prisma (Node TS) React Vite SPA Postgres 16 Docker Compose JS + Python SDKs
Mode	Fetch-on-load SDK evaluation + server fallback Simple predictable rules Auto-provision onboarding
Source	docs/v2/PRD.md V1.1 + locked decisions: fetch-on-load, no SSE, no polling

This PDF was exported from v2/TECH_SPEC.md. The Markdown file remains the source of truth; this PDF is the shareable snapshot for review.

1. Goal + Scope

Build PRD V2 in 4 weeks, solo dev: create flag -> target / rollout % per env -> SDK evaluates on load -> change reflected on next refresh -> audit logged.

In scope: US-1 to US-8 (org/project/env, flag CRUD + kill, % rollout, targeting rules, evaluate/bootstrap, JS+Python SDK, dashboard+playground, light audit). Out of scope: SSE/polling/WebSockets, RBAC, SSO, multivariate, analytics, scheduling, OpenFeature, multi-region — per PRD v2 §8.

Success gates (from PRD v2):

- kill toggle reflected in the 2 demo apps on their next load/bootstrap
- sticky 100% across 1000 evals
- distribution +/-2% at 10k users
- 100% mutations audited
- org->flag setup <2min

2. Architecture

```
[React Vite Dashboard] --JWT--> [Fastify API :3001]
                               | Prisma
[App + SDK] -sdk_key, GET /bootstrap->+-----+
                               |Postgres|  truth, version++
                               +-----+
                               | (optional) Redis 7 snapshot cache
```

Why this:

- Fastify: fastest JSON, low boilerplate for solo.
- Prisma: typed migrations, fast CRUD for `flag_env_state` + `rules`.
- Postgres truth. Redis is **optional** plain read-cache only (TTL snapshot) — no pub/sub, no streams. At <10k eval/min you can drop it entirely.
- Fetch-on-load SDK: `init` -> bootstrap full snapshot once -> eval in-memory <10ms. No background jobs, no connection management, no reconnect logic. A mutation is seen on the next `init`/page load.
- Server `/evaluate` kept for playground + thin clients + debug.

Services (Docker Compose): `api` (Fastify :3001), `web` (:5173 dev / Nginx prod), `postgres:16` (:5432), optional `redis:7` (:6379)

3. Monorepo Layout

```
catalyst/
  docker-compose.yml
  package.json (pnpm workspaces)
  apps/
    api/                                # Fastify + Prisma
      src/
        index.ts                        # fastify bootstrap, cors, jwt
        routes/ auth.ts orgs.ts projects.ts flags.ts rules.ts keys.ts evaluate.ts bootstrap.ts audit.ts pl
        lib/ eval.ts hash.ts cache.ts auth.ts keys.ts audit.ts version.ts
        plugins/ prisma.ts jwt.ts
        prisma/schema.prisma
        Dockerfile
      web/                               # React Vite SPA
        src/ pages/ Flags/ FlagDetail/ Keys/ Playground/ components/ lib/api.ts
  packages/
    js-sdk/ src/index.ts eval.ts (shared with api/lib) storage.ts
    python-sdk/ catalyst/__init__.py eval.py storage.py
  demo/
    js-demo/ python-demo/
  docs/
    v1/ v2/
```

Shared eval logic: `apps/api/src/lib/eval.ts` is source of truth, ported 1:1 to `packages/js-sdk` + `python-sdk`. Same murmur hash + operator table. Tested by shared vectors (`eval.vectors.json`).

No `sse.ts`, no stream gateway, no poll loop anywhere in the repo.

4. Data Model (Prisma)

```
model Organization { id String @id @default(cuid()) name String
  projects Project[] users User[] createdAt DateTime @default(now()) }

model User { id String @id @default(cuid()) email String @unique
  passwordHash String orgId String org Organization @relation(fields:[orgId], references:[id])
  createdAt DateTime @default(now()) }

model Project { id String @id @default(cuid()) orgId String name String
  org Organization @relation(fields:[orgId], references:[id])
  environments Environment[] flags Flag[] apiKeys ApiKey[]
  @@index([orgId]) }

model Environment { id String @id @default(cuid()) projectId String name String // dev|staging|prod
```

```

version Int @default(1) // ++ on any flag change in this env; bootstrap cache-buster (ETag/304)
project Project @relation(fields:[projectId], references:[id])
@@unique([projectId, name]) }

model Flag { id String @id @default(cuid()) projectId String key String
  name String description String? defaultValue Boolean @default(false)
  archived Boolean @default(false) createdAt DateTime @default(now())
  project Project @relation(fields:[projectId], references:[id])
  states FlagEnvState[] rules TargetingRule[]
  @@unique([projectId, key]) }

model FlagEnvState { id String @id @default(cuid()) flagId String env String
  enabled Boolean @default(true) // global kill: false = always defaultValue
  percentage Int @default(0) // 0-100
  version Int @default(1) // ++ per mutation
  flag Flag @relation(fields:[flagId], references:[id], onDelete:Cascade)
  @@unique([flagId, env]) @@index([flagId]) }

model TargetingRule { id String @id @default(cuid()) flagId String env String
  priority Int // 0 = first, drag-reorder re-normalizes 0..n
  conditionsJson Json // [{attr, op, value}]
  serve Boolean // true/false to serve on match
  flag Flag @relation(fields:[flagId], references:[id], onDelete:Cascade)
  @@index([flagId, env, priority]) }

model ApiKey { id String @id @default(cuid()) projectId String
  env String name String prefix String // cp_dev_xxxx for lookup
  hash String // sha256(full key), never store raw
  revoked Boolean @default(false) createdAt DateTime @default(now())
  project Project @relation(fields:[projectId], references:[id])
  @@index([prefix]) @@index([projectId, env]) }

model AuditLog { id String @id @default(cuid()) orgId String projectId String
  flagId String? env String? actor String // user email or sdk:prefix
  action String // flag.created|toggled|rollout.changed|rule.created|...|key.rotated
  before Json? after Json? createdAt DateTime @default(now())
  @@index([projectId, flagId]) @@index([createdAt]) }

```

Rules:

- `flag.key` regex `^[a-z0-9-_.]+`, unique per `projectId` (env isolation lives in `FlagEnvState`, not duplicate flags).
- `Environment.version` is the bootstrap cursor + ETag. Any flag/rule/state change in `project+env` -> `env.version++` + `state.version++`. Used only so a reload can send `If-None-Match` and get 304 if nothing changed.
- No edit/delete on `AuditLog` (no API, Prisma middleware blocks).

Indexes cover: flag list by project, state lookup by flag+env, bootstrap by project+env, audit timeline by flag.

5. API Contract (all JSON, base `/api/v1`)

Auth:

- Dashboard: POST `/auth/signup` {email,password,orgName} -> auto-creates org + project "default" + envs dev/staging/prod + 3 sdk_keys -> sets httpOnly JWT cookie. Returns {user, org, project, envs}. Setup <2min gate.
- POST `/auth/login`, GET `/auth/me`, POST `/auth/logout`.
- SDK: header `x-sdk-key: cp_<env>_<random32>`. Lookup by prefix -> sha256 compare -> scope to `projectId+env`. Read-only: only bootstrap/evaluate/playground-eval. No write.

Flags:

- GET /projects/:pid/flags?search=&archived=false&env=prod -> [{key, name, enabled, percentage, prodSummary}]
- POST /projects/:pid/flags {key, name, description, defaultValue} -> 409 on dup key, 400 on bad regex. Creates 3 FlagEnvState rows (one per env, enabled=true, percentage=0) + audit.
- PATCH /flags/:id {name, description, defaultValue} POST /flags/:id/archive POST /flags/:id/unarchive
- PUT /flags/:id/state/:env {enabled, percentage} -> validate 0-100 int -> version++, env.version++, audit.

Rules:

- GET /flags/:id/rules?env=prod ordered by priority
- POST /flags/:id/rules {env, conditions:[{attr, op, value}], serve} PATCH /rules/:rid DELETE /rules/:rid PUT /flags/:id/rules/reorder {env, orderedIds:[...]} -> renormalize priority 0..n, bump versions, audit.

Keys:

- GET /projects/:pid/keys -> [{env, prefix, createdAt, revoked}] (never raw)
- POST /projects/:pid/keys/rotate {env} -> revoke old, return new raw key once + audit.

Evaluate (server fallback + playground):

- POST /evaluate header x-sdk-key or JWT, body {flagKey, context:{userId?, email?, country?, version?}} -> {value, reason, version} reason = disabled|rule:<id>|rollout|default. Missing context field -> that condition safely skips (no crash).

Bootstrap (SDK init, fetch-on-load):

- GET /bootstrap?projectId=&env=prod header x-sdk-key -> {version: envVersion, flags:[{key, enabled, percentage, defaultValue, version, rules:[{id, priority, conditions, serve}]}]} Archived excluded. **ETag = version; SDK sends If-None-Match -> 304 when unchanged.** SDK caches to disk so a reload (even offline) works from cache.
- **No GET /stream, no SSE, no push of any kind.** A toggle is reflected on the next bootstrap.

Audit + Playground:

- GET /flags/:id/audit?env= -> [{timestamp, actor, action, before, after}] desc.
- POST /flags/:id/test-eval {env, context} (JWT) -> {value, reason, matchedRuleId} for dashboard playground.

Errors: {error:{code, message, details}} codes

VALIDATION|NOT_FOUND|CONFLICT|UNAUTHORIZED|RATE_LIMITED. p95 server eval <150ms (Redis-cached snapshot optional).

6. Evaluation Engine (shared, simple predictable)

Order (PRD v2 §5):

1. if archived -> defaultValue, reason=default
2. if !enabled (kill OFF) -> defaultValue, reason=disabled
3. rules by priority asc -> first where ALL conditions true -> serve, reason=rule:<id>
4. rollout: if userId present and hash%100 < percentage -> true, reason=rollout
else if userId missing -> skip rollout -> defaultValue
5. else defaultValue, reason=default (or rollout when percentage 100/0 covers non-targeted)

Hash: `murmur3_32(flagKey + ":" + userId) % 100 < percentage`. Same murmur seed (0) in TS + Python. Sticky 100%. 0% = OFF, 100% = ON for non-targeted with `userId`.

Operators (AND within rule, OR across rules fixed — no configurable toggle in V1): `equals`, `notEquals`, `in`, `notIn`, `endsWith`, `startsWith`, `contains`, `gte`, `lt`, `exists`, `notExists` Semantics (simple):

- All compares are case-sensitive strings except `gte/lt` which try numeric compare first, else lexicographic string compare. `version>=2.0` = naive string/numeric, NOT semver (documented limit).
- Missing attr -> condition false -> rule skips (never throws).
- `in/notIn` value must be array.
- `exists` value boolean.

TS pseudocode (`apps/api/src/lib/eval.ts`):

```
export function evaluate(snapshot: FlagSnapshot, ctx: Ctx) {
  if (!snapshot.enabled) return {value: snapshot.defaultValue, reason: 'disabled'};
  for (const r of sortBy(snapshot.rules, 'priority'))
    if (r.conditions.every(c => match(c, ctx))) return {value: r.serve, reason: `rule:${r.id}`};
  if (ctx.userId) {
    const b = murmur32(`${snapshot.key}:${ctx.userId}`) % 100;
    if (b < snapshot.percentage) return {value: true, reason: 'rollout'};
  }
  return {value: snapshot.defaultValue, reason: 'default'};
}
```

7. Caching, Versioning, Propagation (fetch-on-load)

- Read path: `evaluate/bootstrap` -> optional Redis `snap:project:{pid}:env:{env}` (TTL 60s) -> miss loads Postgres, caches. No Redis? Read straight from Postgres (fine at V1 scale).
- Write path: Prisma txn -> update state/rule + `env.version++` + `state.version++` + `audit` -> invalidate `snap:*`. **Nothing is pushed to clients.**
- SDK flow: page load -> init -> bootstrap (If-None-Match on version -> 304 skip) -> cache mem+disk -> eval local -> done. Offline -> last cache -> `defaultValue`. Never throw.
- Client refresh semantics made explicit: a changed toggle is observed on the **next** init/page load. Documented, not a bug.

8. Auth + Keys Detail

- Password `bcrypt(12)`, JWT HS256 7d expiry in `httpOnly SameSite=Lax` cookie. All dashboard routes require JWT + `orgId` match (tenant isolation check on every query).
- `sdk_key` format `cp_{env}_{32hex}`, stored sha256. Prefix indexed for fast lookup. Rotation revokes old immediately; SDK re-inits with new key.
- V1 uses single `sdk_key` per env read-only. No write via SDK key.
- Env switch never leaks: every query filters `projectId` + `env`.

9. Dashboard (React Vite SPA)

Routes: `/login /signup /:orgId/flags /:orgId/flags/:key (tabs dev|staging|prod) /:orgId/keys`

- Flag list: search + status dot (green ON / gray OFF / amber partial %) + per-env summary + kill toggle inline.
- Flag detail: top Kill switch, Rollout slider + numeric, Rules list + Add modal (attr/op/value/serve) + drag reorder (dnd-kit), Playground inputs -> `value + reason + matchedRule`, History timeline (read-only).
- Keys page: per-env copy/rotate with confirm + show-once.
- UX rule: every control surfaces `reason`. Non-dev toggle <30s.
- API client `lib/api.ts` uses `fetch` with credentials include, Zod validation.

10. SDKs

JS (packages/js-sdk):

```
await catalyst.init({sdkKey, baseUrl, env, projectId}); // fetch-on-load bootstrap
catalyst.isEnabled('new-checkout', {userId:'u123', email:'a@test.com'});
```

- Node + browser (fetch, localStorage fallback, in-memory primary). No EventSource, no polling, no `onUpdate`. Re-init (or page reload) to get fresh values.

Python (packages/python-sdk):

```
catalyst.init(sdk_key=..., base_url=..., env="prod", project_id=...)
catalyst.is_enabled("new-checkout", {"userId":"u123"})
```

- `requests`, disk cache `~/.catalyst/cache.json`, never raises on eval (returns default). No background thread (nothing to poll).

Both share `eval.vectors.json` tests.

11. Testing + Perf

- `vitest` (api + js-sdk) + `pytest`: hash stickiness (same user 1000 evals same), distribution +/-2% at 10k synthetic userIDs, all operators + missing-field skips, kill precedence, reorder priority, archived excluded, audit on every mutation, tenant isolation (env A change never appears in B bootstrap), bootstrap 304 when version unchanged.
- k6 smoke (optional Wk4): 100 RPS evaluate p95 <150ms local.
- Manual demo gate: toggle prod OFF -> reload both demo apps -> both show new value with correct `reason`.

12. Build Order (4 weeks, solo)

- **Wk1 Auth+CRUD:** Compose + Prisma migrate + signup auto-provision + flag CRUD + `FlagEnvState` + keys + audit hook + web shell + flag list.
- **Wk2 Engine:** `eval.ts` + hash + operators + `/evaluate` + reorder + vectors tests + playground API.
- **Wk3 Bootstrap+JS:** `/bootstrap` (ETag/304) + js-sdk (`init/isEnabled/cache`) + js-demo + flag detail UI (kill/slider/rules).
- **Wk4 Python+Polish:** python-sdk parity + history/keys UI + 2 demo apps + audit timeline + README + demo video. Stretch: live deploy (Render + Vercel).

Start command Day 1:

```
docker compose up -d postgres # redis optional
pnpm --filter api prisma migrate dev --name init
pnpm dev
```

13. Risks -> Mitigations

Stale values after a toggle -> next load/bootstrap picks them up; documented behavior, not silent. Hash skew -> murmur + distribution test. Key leak -> prefix lookup + rotate + revoke immediate. Version string confusion -> document naive compare, semver Phase 2.